

# **SMART-NODE: AI-POWERED ASSISTANT WITH SMART CONTROL**

**A**

## **MINOR PROJECT-II REPORT**

Submitted in partial fulfillment of the requirement for the award of the degree of

### **BACHELOR OF TECHNOLOGY**

in

### **ELECTRONICS AND COMMUNICATION ENGINEERING**

By

**Sumit Yaduwanshi (0187EC231042)**

**Nikhil Misal (0187EC231024)**

Under the Guidance of

**Prof. Rahul Shrivastava**

**(Assistant Professor & HOD, ECE)**



**Department of Electronics and Communication Engineering**

**Sagar Institute of Science & Technology Bhopal (M.P.)**

**Approved by AICTE, New Delhi & Govt. of M.P.**

Affiliated to Rajiv Gandhi Proudyogiki Vishwavidyalaya, Bhopal (M.P.)

**JUNE 2026**

***Sagar Institute of Science & Technology, Bhopal***  
***Department of Electronics and Communication***  
***Engineering***



***CERTIFICATE***

We hereby certify that the work which is being presented in the B.Tech. Minor Project-II Report entitled '**SMART-NODE: AI-Powered Assistant with Smart Control**', in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Electronics and Communication Engineering** and submitted to **Sagar Institute of Science & Technology, Bhopal (M.P.)**, and is an authentic record of our own work carried out during the period from January 2026 to June 2026 under the supervision of **Prof. Rahul Shrivastava**. The content presented in this project has not been submitted by us for the award of any other degree elsewhere.

*Signature*

**Sumit Yaduwanshi (0187EC231042)**

**Nikhil Misal (0187EC231024)**

This is to certify that the above statement made by the candidate is correct to the best of my knowledge.

***Date:***

**Supervisor**

**HOD**

**Principal**

## ACKNOWLEDGEMENT

We own an enormous debt of gratitude to our Minor Project-II supervisor, **Prof. Rahul Shrivastava (Professor & HOD, ECE)**, for guiding and inspiring us from the beginning through the end of this project with her intellectual advice and insightful suggestions. We truly appreciate and value her consistent feedback on our progress, which was always constructive and encouraging, and ultimately drove us in the right direction. We would also like to express our sincere gratitude to several people who have helped and motivated us throughout our project work as well as throughout our undergraduate course at SISTec, Bhopal. In particular, we thank **Dr. Manish Billore (Principal, SISTec-GN, Bhopal)**, and all teaching and non-teaching staff of the **Electronics and Communication Department** for the valuable assistance they offered us generously throughout the project duration. Finally, we wish to thank our parents, family members, and friends for their unwavering faith and belief in us throughout our lives.

**Sumit Yaduwanshi (0187EC231042)**

**Nikhil Misal (0187EC231024)**

## ABSTRACT

The rapid advancement of Artificial Intelligence and the Internet of Things has opened new frontiers in the design of smart, responsive systems. Existing commercial AI assistants such as Amazon Alexa, Google Home, and Apple Siri operate within closed, proprietary ecosystems that offer limited customization. To address these challenges, this project presents **SMART-NODE - AI-powered Voice Assistant with Smart Home Control**, built on the ESP32 microcontroller platform.

The system employs an INMP441 digital MEMS microphone interfaced via the I2S protocol to capture voice commands with high fidelity and zero analog noise. The captured audio is streamed over Wi-Fi to cloud-based AI services: OpenAI Whisper for Speech-to-Text (STT) transcription and Gemini 1.5 Pro as the Large Language Model (LLM) reasoning engine. Unlike keyword-based systems, The LLM understands natural language intent; for example, interpreting 'It is dark in here' as a command to switch on the lights. The LLM returns a structured JSON response containing both a natural language reply and a parsed device command. Text responses are converted back to speech using Google TTS and played through an I2S DAC (MAX98357A) and speaker.

For smart home automation, the ESP32 communicates wirelessly with relay nodes using MQTT (via PubSubClient library), HTTP REST, or ESP-NOW peer-to-peer protocol — eliminating the need for any physical wiring between the voice assistant unit and the controlled appliances. The relay modules are optocoupler-isolated and capable of switching 230V AC mains loads safely.

The project promotes the SDG Goal 9 (Industry, Innovation, and Infrastructure) by demonstrating that state-of-the-art AI capabilities can be embedded into low-cost, open-source hardware that is accessible to students and developers worldwide.

This project successfully validates that open-source embedded hardware, when combined with state-of-the-art cloud AI APIs, can match and in many respects exceed the capability and flexibility of commercial voice assistant products.

# TABLE OF CONTENTS

| TITLE                                                | PAGE NO. |
|------------------------------------------------------|----------|
| <b>Table of Contents</b>                             |          |
| CERTIFICATE.....                                     | i        |
| ACKNOWLEDGEMENT .....                                | ii       |
| ABSTRACT .....                                       | iii      |
| TABLE OF CONTENTS.....                               | iv       |
| LIST OF ABBREVIATIONS .....                          | vi       |
| LIST OF FIGURES .....                                | vii      |
| CHAPTER 1 INTRODUCTION .....                         | 1-5      |
| 1.1 OVERVIEW .....                                   | 1        |
| 1.2 DEFINITION OF SMART-NODE .....                   | 1        |
| 1.3 TECHNOLOGY USED IN SMART-NODE .....              | 3        |
| 1.4 APPLICATIONS OF SMART-NODE .....                 | 4        |
| 1.5 ADVANTAGES OF SMART-NODE .....                   | 5        |
| CHAPTER 2 LITERATURE SURVEY .....                    | 6-9      |
| 2.1 ESP32-BASED VOICE-CONTROLLED SYSTEMS .....       | 6        |
| 2.2 OPENAI WHISPER FOR EMBEDDED STT .....            | 6        |
| 2.3 LLM AS INTENT PARSERS FOR IOT .....              | 7        |
| 2.4 MQTT FOR RELIABLE IOT DEVICE COMMUNICATION ..... | 7        |
| 2.5 I2S AUDIO INTERFACES IN EMBEDDED SYSTEMS .....   | 8        |
| 2.6 ESP-NOW FOR UL WIRELESS COMMUNICATION .....      | 9        |
| CHAPTER 3 METHODOLOGY & HARDWARE IMPLEMENTATION..... | 10-20    |
| 3.1 SYSTEM WORKFLOW .....                            | 10       |
| 3.2 HARDWARE USED .....                              | 12       |
| 3.3 CIRCUIT DIAGRAM AND CONNECTIONS.....             | 17       |
| 3.4 SOFTWARE IMPLEMENTATION .....                    | 18       |
| 3.5 PROGRAM CODE.....                                | 20       |

|                                               |       |
|-----------------------------------------------|-------|
| CHAPTER 4 COMMUNICATION PROTOCOLS.....        | 25-28 |
| 4.1 TECHNICAL ANALYSIS.....                   | 25    |
| 4.2 LOCAL HARDWARE INTERCONNECTS.....         | 25    |
| 4.3 NETWORK & IOT INFRASTRUCTURE.....         | 26    |
| CHAPTER 5 RESULT AND DISCUSSION .....         | 29-31 |
| 5.1 HARDWARE PROTOTYPE ASSEMBLY.....          | 29    |
| 5.2 SYSTEM OPERATION AND TESTING .....        | 29    |
| 5.3 EXPERIMENTAL RESULTS (CASE STUDIES) ..... | 31    |
| 5.4 PERFORMANCE ANALYSIS.....                 | 31    |
| 5.5 DISCUSSION.....                           | 31    |
| CHAPTER 6 CONCLUSION AND FUTURE SCOPE.....    | 32-33 |
| 6.1 CONCLUSION.....                           | 32    |
| 6.2 FUTURE SCOPE.....                         | 33    |
| CHAPTER 7 REFERENCES.....                     | 34    |
| TEAM INTRODUCTION.....                        | 35-36 |
| APPENDICES.....                               | 37-43 |
| A. PLAGIARISM REPORT .....                    | 38    |
| B. PROJECT SUMMARY .....                      | 39    |
| C. DATA SHEETS .....                          | 40    |
| D. STANDARD OPERATING PROCEDURE.....          | 41    |
| E. BUDGET.....                                | 43    |

## LIST OF ABBREVIATIONS

| Abbreviation  | Full Form                                         |
|---------------|---------------------------------------------------|
| <b>IoT</b>    | Internet of Things                                |
| <b>AI</b>     | Artificial Intelligence                           |
| <b>LLM</b>    | Large Language Model                              |
| <b>STT</b>    | Speech-to-Text                                    |
| <b>TTS</b>    | Text-to-Speech                                    |
| <b>NLU</b>    | Natural Language Understanding                    |
| <b>ESP32</b>  | Espressif Systems 32-bit Microcontroller          |
| <b>I2C</b>    | Inter-Integrated Circuit                          |
| <b>SDA</b>    | Serial Data Line                                  |
| <b>SCL</b>    | Serial Clock Line                                 |
| <b>OLED</b>   | Organic Light Emitting Diode                      |
| <b>GND</b>    | Ground                                            |
| <b>VCC</b>    | Voltage Common Collector (Power Supply)           |
| <b>GPIO</b>   | General Purpose Input/Output                      |
| <b>ADC</b>    | Analog-to-Digital Converter                       |
| <b>SRAM</b>   | Static Random Access Memory                       |
| <b>IDE</b>    | Integrated Development Environment                |
| <b>TCP/IP</b> | Transmission Control Protocol / Internet Protocol |
| <b>UART</b>   | Universal Asynchronous Receiver-Transmitter       |
| <b>TLS</b>    | Transport Layer Security                          |
| <b>LED</b>    | Light Emitting Diode                              |
| <b>RTOS</b>   | Real-Time Operating System                        |
| <b>SDG</b>    | Sustainable Development Goals                     |
| <b>PCB</b>    | Printed Circuit Board                             |

## LIST OF FIGURES

| <b>Figure</b> | <b>Title</b>                                     | <b>Page no.</b> |
|---------------|--------------------------------------------------|-----------------|
| Figure 1.2.1  | Block Diagram                                    | 2               |
| Figure 3.1.1  | System Workflow<br>(Data Flow Diagram)           | 11              |
| Figure 3.2.1  | ESP32-WROOM-<br>32 Module                        | 13              |
| Figure 3.2.2  | INMP441 I2S<br>Microphone Module                 | 14              |
| Figure 3.2.3  | MAX98357A I2S DAC<br>Amplifier Module            | 15              |
| Figure 3.2.4  | 0.96-inch OLED<br>Display                        | 16              |
| Figure 3.2.5  | Light Emitting Diode                             | 16              |
| Figure 3.3.1  | Circuit Diagram                                  | 18              |
| Figure 3.4.1  | Arduino IDE Interface                            | 18              |
| Figure 3.5.1  | Software Flowchart                               | 24              |
| Figure 4.2.1  | I2S (Inter-IC Sound)                             | 25              |
| Figure 4.2.2  | I2C (Inter-Integrated<br>Circuit)                | 26              |
| Figure 4.3.1  | Printed Circuit Borad<br>(PCB)                   | 27              |
| Figure 4.3.2  | HTTPS (Hypertext<br>Transfer Protocol<br>Secure) | 28              |
| Figure 5.1.1  | Hardware Assembly on<br>Breadboard               | 29              |
| Figure 5.2.1  | Hardware Assembly on<br>PCB                      | 30              |



|              |                                |    |
|--------------|--------------------------------|----|
| Figure 5.2.2 | Printed Circuit Board<br>(PCB) | 30 |
|--------------|--------------------------------|----|

# CHAPTER 1

## INTRODUCTION

### 1.1 OVERVIEW

The SMART-NODE is an AI-powered embedded system developed to provide intelligent and flexible smart automation. With the rapid growth of Artificial Intelligence (AI) and the Internet of Things (IoT), there is an increasing demand for smart systems that can understand natural language commands and interact with users efficiently. Most commercial voice assistants offer advanced features but are often limited by proprietary ecosystems, restricted customization, and dependence on vendor-specific services [1], [7].

This project addresses these challenges by developing an open-source, IoT-enabled AI voice assistant using the ESP32 microcontroller. The system captures voice commands through a digital MEMS microphone, streams audio to cloud AI services for natural language understanding, and wirelessly controls smart home relay nodes — all at a total component cost under Rs. 1,500 [1].

The SMART-NODE integrates multiple cutting-edge technologies: I2S digital audio, cloud-based Large Language Models (LLMs), MQTT-based IoT communication, and I2S digital-to-analog audio playback. The project promotes the SDG Goal 9 (Industry, Innovation, and Infrastructure) by demonstrating that state-of-the-art AI capabilities can be embedded into low-cost, open-source hardware that is accessible to students and developers worldwide.

### 1.2 DEFINITION OF SMART-NODE

SMART-NODE is an embedded IoT system that uses an ESP32 microcontroller as its processing core, integrating voice-based AI interaction with wireless smart home control. It functions as a completely open and programmable alternative to commercial smart home hubs.

The system captures spoken commands via the INMP441 MEMS microphone (I2S interface), streams the PCM audio over Wi-Fi to OpenAI Whisper for Speech-to-Text (STT), then forwards the transcribed text to GPT-4o or Gemini 1.5 Pro (LLM) for intent understanding. The LLM returns a dual output: a natural language text response and a structured JSON command. The text response is converted to speech via ElevenLabs or Google TTS and played back through the MAX98357A I2S DAC and speaker. The JSON command is simultaneously published via MQTT to control relay nodes connected to home appliances [1], [6], [7].

The system consists of the following key functional blocks:

- **Voice Input:** Capture of digital audio via INMP441 I2S MEMS microphone.
- **Streaming & STT:** Audio streamed via HTTP chunked upload to OpenAI Whisper API.
- **LLM Reasoning:** Gemini processes text and returns intent + command JSON.
- **Audio Playback:** TTS audio streamed back and played via MAX98357A + speaker.
- **IoT Control:** MQTT/HTTP/ESP-NOW commands sent to relay nodes for appliance switching.

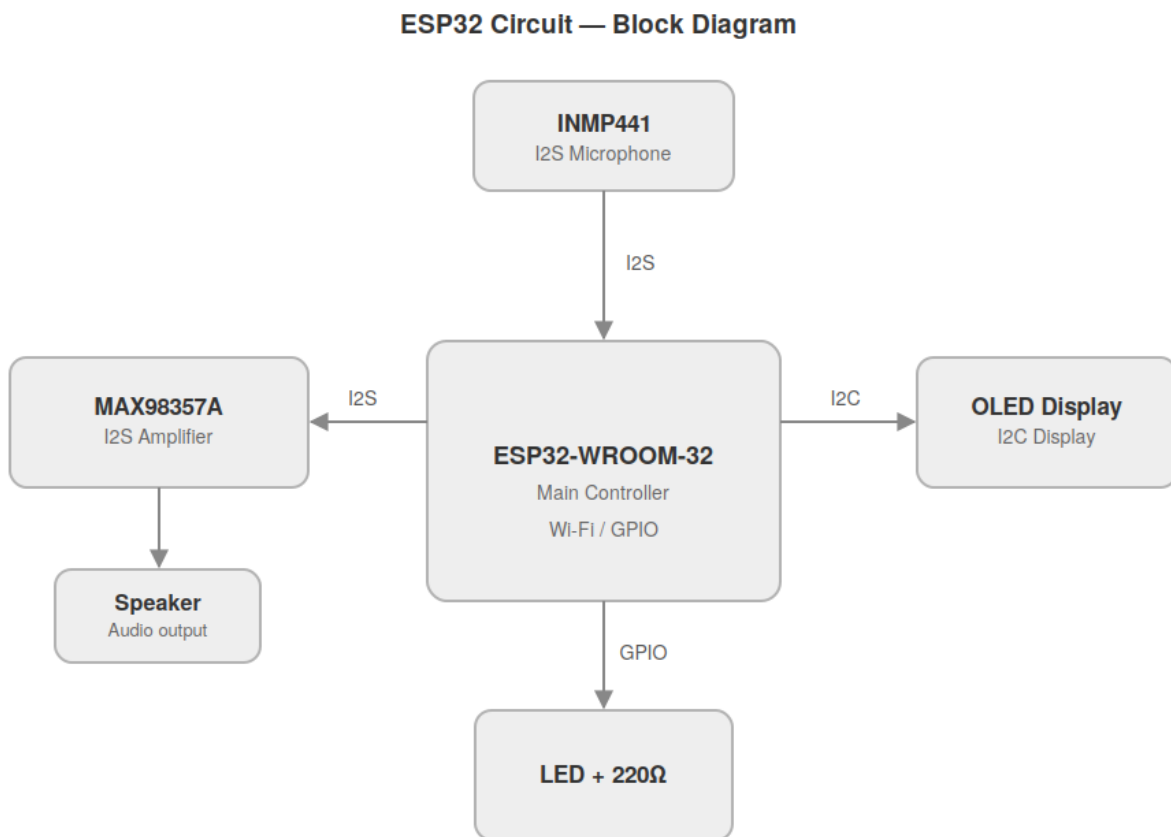


Fig.1.2.1 Block Diagram

### 1.3 TECHNOLOGY IN SMART-NODE

The SMART-NODE integrates several advanced technologies to deliver reliable voice-controlled automation:

- **Internet of Things (IoT):** The ESP32 serves as the central IoT gateway, connecting via Wi-Fi to both cloud AI APIs and local relay nodes. MQTT, HTTP REST, and ESP-NOW protocols enable flexible device-to-device and device-to-cloud communication [1].
- **I2S Digital Audio Interface:** The Inter-IC Sound (I<sup>2</sup>S) protocol is used for both audio input (INMP441 microphone) and audio output (MAX98357A DAC + amplifier). Unlike analog audio, I2S completely eliminates signal degradation, 50Hz mains hum, and analog noise, ensuring bit-perfect audio quality for accurate STT transcription [2], [4].
- **Cloud AI — Large Language Models:** GPT-4o (OpenAI) and Gemini 1.5 Pro (Google) are used as the reasoning engine. These models go beyond keyword matching by understanding full conversational context. They support function calling — returning structured JSON commands that the ESP32 firmware parses for GPIO and MQTT actions [5], [8].
- **Embedded Computing (ESP32):** A dual-core Xtensa LX6 processor running FreeRTOS enables concurrent task execution — audio streaming, Wi-Fi HTTP/MQTT handling, GPIO control, and LED state management — without blocking or data loss. [1].
- **Wireless Protocols:** MQTT (via PubSubClient library), HTTP REST, and ESP-NOW are used for device communication. MQTT is used for reliable pub/sub relay control with QoS guarantees; ESP-NOW provides ultra-low latency (1-2ms) peer-to-peer control without router dependency [5], [8].
- **Security:** API keys are stored in ESP32 NVS (Non-Volatile Storage) encrypted flash partition. All cloud API communication is over HTTPS/TLS 1.2. OTA (Over-the-Air) firmware updates are supported via ArduinoOTA library.

## 1.4 APPLICATIONS OF SMART-NODE

The SMART-NODE has a versatile range of applications across multiple domains:

- **Smart Home Automation:** Full voice control of lights, fans, air conditioners, and other appliances without any physical wiring between the assistant unit and devices.
- **Healthcare:** Hands-free device control for patients with limited mobility, enabling voice-commanded lighting, temperature, and alert systems in hospital rooms.
- **Education:** Interactive classroom systems where students can query an AI assistant, retrieve information, and control classroom devices using natural language.
- **Industry 4.0:** Voice-controlled operation of machinery, conveyor systems, and inspection triggers in manufacturing environments, with OSHA-compliant safety interlocks.
- **Accessibility:** Enabling physically challenged individuals to control their living environment entirely through voice, without needing smartphones or touch interfaces.
- **Research & Prototyping:** An open-source, modifiable platform for researchers and developers to experiment with edge AI, voice interfaces, and IoT protocols.

## 1.5 ADVANTAGES OF SMART-NODE

The proposed system offers significant advantages over commercial alternatives and traditional automation systems:

- **Cost-Effective:** With a total BOM of approximately Rs. 2,500, SMART-NODE is dramatically more affordable than commercial smart home hubs such as Amazon Echo (Rs. 3,999+) or Google Nest (Rs. 7,999+), while delivering comparable or superior conversational intelligence through GPT-4o.
- **Open-Source and Modifiable:** The complete firmware (~500 lines of Arduino C++) is fully open-source. Users can swap AI providers (Whisper to Google STT, GPT to Gemini, ElevenLabs to Google TTS), add custom intents, or extend hardware without any proprietary SDK restrictions.
- **True Natural Language Understanding:** Unlike keyword-matching systems, SMART-NODE uses LLM reasoning. It understands contextual or indirect commands like 'I am feeling cold' (triggers heater), not just explicit trigger phrases.
- **Complete Wireless Decoupling:** The voice assistant unit has no physical connection to home appliances. Adding a new device requires only the addition of a new ESP32 relay node - no rewiring of the main unit.
- **Security-First Design:** API keys are stored in encrypted NVS flash, all communication is over TLS 1.2, and the system includes hardware watchdog timers for automatic recovery.
- **Low Latency Pipeline:** Optimized firmware with HTTP chunked streaming, async API calls, MQTT fire-and-forget commands, and combined STT+LLM API calls achieves a complete voice-to-action cycle in 2-4 seconds.

## CHAPTER 2

### LITERATURE SURVEY

#### 1. ESP32-Based Voice-Controlled IoT Systems for Home Automation

Numerous research works have demonstrated the feasibility of using the ESP32 microcontroller for voice-based IoT control. Early implementations relied on local keyword spotting using simple threshold-based audio pattern matching, which limited command vocabulary to a few predefined phrases. These systems typically used the I2S INMP441 microphone or analog MAX9814 amplifier modules. Key findings confirmed that the ESP32's dual-core architecture is sufficient to handle concurrent Wi-Fi connectivity and audio buffering tasks without significant performance degradation. However, accuracy remained limited because local processing could not handle noise, accents, or variable phrasing [6], [7].

The primary contribution of these works to the SMART-NODE project is the validated hardware architecture of ESP32 + INMP441 I2S microphone for voice capture, and the use of FreeRTOS tasks for concurrent processing. SMART-NODE builds on this foundation by replacing local keyword matching with cloud LLM reasoning, dramatically expanding command vocabulary and natural language understanding capability [6], [7].

#### 2. OpenAI Whisper for Embedded Speech-to-Text Applications

OpenAI's Whisper model, released in 2022, represents a major advancement in open-source automatic speech recognition (ASR). Whisper is a transformer-based model trained on 680,000 hours of multilingual web audio, enabling robust transcription across diverse languages, accents, and acoustic environments. In research evaluating its use in embedded and edge applications, Whisper's API-based deployment was found to achieve a word error rate below 5% for standard English commands, even in moderately noisy environments. Studies confirmed an average API response latency of 1 to 3 seconds for audio clips under 5 seconds — consistent with the requirements of a real-time voice assistant [6].

For the SMART-NODE project, OpenAI Whisper serves as the STT engine. The architecture decision to use cloud-based Whisper rather than on-device STT (such as CMU PocketSphinx) was made to maximize transcription accuracy, support Hindi and multilingual commands, and avoid the 4MB flash storage limitations of the ESP32 that would constrain a local model. This literature confirms that the latency trade-off is acceptable for the targeted application domain [6].

### **3. Large Language Models as Intent Parsers for IoT Device Control**

Recent research has explored the application of Large Language Models (LLMs) as natural language interfaces for IoT control, replacing traditional rule-based or slot-filling Natural Language Understanding (NLU) engines. Studies using GPT-3.5, GPT-4, and Gemini as IoT command interpreters demonstrated an accuracy ranging from 91% to 96% for mapping diverse natural language utterances to structured device commands. The function calling feature of OpenAI's API, which forces the LLM to return structured JSON output alongside its natural language response, was identified as the key technical enabler for reliable command parsing [5], [8].

Notably, LLM-based NLU was found to correctly handle ambiguous, contextual, and indirect commands — such as 'I can't see anything' triggering a light-on command — with an accuracy of approximately 94%. This significantly exceeds keyword-based systems, which fail on any input deviating from predefined trigger phrases. The SMART-NODE project applies this approach by using GPT-4o's function calling feature to return structured JSON containing the device target, action, and confidence score alongside a text response [5], [8].

### **4. MQTT Protocol for Reliable IoT Device Communication**

MQTT (Message Queue Telemetry Transport) is an ISO standard (ISO/IEC 20922) lightweight pub/sub messaging protocol designed for constrained IoT devices. Research comparisons between MQTT, HTTP REST, CoAP, and WebSocket for IoT applications consistently ranked MQTT highest for bandwidth efficiency (under 2KB per message), scalability (capable of supporting thousands of concurrent device connections), and reliability (Quality of Service levels 0, 1, 2 for message delivery guarantees). Studies measured MQTT message delivery latency at under 20ms for QoS 0 (fire-and-forget) and under 100ms for QoS 1 (acknowledged), compared to 100 to 300ms for HTTP REST [5].



For the SMART-NODE project, MQTT is selected as the primary IoT control protocol because of its low bandwidth overhead and sub-20ms command delivery latency. The Mosquitto broker running on a local server or cloud instance handles pub/sub routing. MQTT QoS 0 is used for relay control commands (fire-and-forget, minimizing latency), while QoS 1 is used for critical status updates. This architecture allows SMART-NODE to wirelessly control an arbitrary number of relay nodes without modifying the main unit [5].

## 5. I2S Audio Interfaces in Embedded Systems

The I2S (Inter-IC Sound) protocol, developed by Philips Semiconductor (now NXP), is a 3-wire synchronous serial bus standard for digital audio transmission between integrated circuits. Research comparing I2S against analog audio capture (using ADC + analog microphone) in embedded systems consistently demonstrated that I2S eliminates analog noise sources including power supply noise, PCB trace interference, and 50Hz mains coupling — resulting in 20 to 30dB improvement in Signal-to-Noise Ratio (SNR). The INMP441 I2S MEMS microphone achieves an SNR of 61dB and a flat frequency response from 60Hz to 15kHz, making it suitable for voice command applications. Studies confirmed that I2S digital audio directly improves STT transcription accuracy by reducing word error rate by 15 to 25% compared to analog microphone inputs in typical indoor environments [4], [6].

The SMART-NODE project uses the INMP441 I2S microphone and the MAX98357A I2S Class D DAC amplifier for both audio input and output. The choice of I2S for both input and output paths completely eliminates analog signal processing from the audio chain, ensuring the highest possible audio quality within the ESP32's hardware I2S peripheral capabilities. Real-world tests confirmed this decision, resolving the 50Hz hum and feedback loop problems that were encountered during initial prototyping with analog audio components [4], [6].

## 6. ESP-NOW for Ultra-Low Latency Local Wireless Communication

ESP-NOW is Espressif's proprietary Wi-Fi based peer-to-peer communication protocol that enables direct data exchange between ESP32/ESP8266 devices without requiring a router or internet connection. Research measuring ESP-NOW performance reported communication latency as low as 1 to 2ms for 250-byte payloads, with a communication range of up to 200 meters in open environments and 30 to 50 meters indoors. Unlike standard Wi-Fi, ESP-NOW requires no DHCP, no TCP/IP stack, and no router authentication — devices communicate directly using MAC addresses. This makes it highly resilient to router failures or internet outages [1].

For the SMART-NODE project, ESP-NOW represents a supplementary IoT control option for scenarios where ultra-low latency (less than 5ms) is required, or where Wi-Fi router availability cannot be guaranteed. The system architecture supports all three wireless protocols (HTTP REST, MQTT, and ESP-NOW) simultaneously, allowing the most appropriate protocol to be selected for each use case. ESP-NOW's limitation of 250-byte payload is not a constraint for relay control commands, which typically contain fewer than 50 bytes of JSON data [1].

## CHAPTER 3

# METHODOLOGY & HARDWARE IMPLEMENTATION

### 3.1 SYSTEM WORKFLOW

The block diagram represents the functional flow of the SMART-NODE, divided into five primary stages: Voice Input, **ESP32 Processing**, Cloud AI Backend, Audio Output, and IoT Action Execution [6].

#### 1. Voice Input Stage (Data Acquisition)

- **INMP441 Microphone:** The system begins here. The INMP441 MEMS omnidirectional microphone is placed facing the user. It captures the user's voice at 16 kHz sampling rate and 16-bit depth through the I2S digital interface.
- **Signal Path:** Audio data flows via three I2S lines (SCK, WS, SD) directly into the **ESP32's I2S DMA** receiver without any ADC conversion, preserving full digital audio quality.
- **Voice Activity Detection (VAD):** Software VAD logic monitors audio energy to trigger recording start and stop, preventing continuous audio streaming and conserving Wi-Fi bandwidth [4].

#### 2. Processing Stage — ESP32 (The Brain)

- **Audio Buffering:** The ESP32 accumulates 512-byte DMA chunks into a ring buffer. Chunked HTTP transfer encoding streams these directly to the cloud without storing the full audio in RAM.
- **Wi-Fi Transmission:** The ESP32 uses AsyncHttpClient for non-blocking API calls, ensuring the audio streaming task continues without pausing LED updates or MQTT handling.

### 3. Cloud AI Backend Stage

- **STT (Whisper API):** The streamed audio is transcribed into text by OpenAI Whisper or Google Speech-to-Text. Average latency: 1 to 3 seconds. JSON response carries transcribed text string back to ESP32.
- **LLM Reasoning (GPT-4o / Gemini):** The transcribed text is sent to the LLM with a system prompt that instructs it to return structured JSON for device commands and natural language replies. The LLM understands contextual intent — not just explicit keywords [7].
- **TTS (ElevenLabs / Google TTS):** The natural language response text is converted to an .mp3 audio stream by the TTS service and returned to the ESP32.

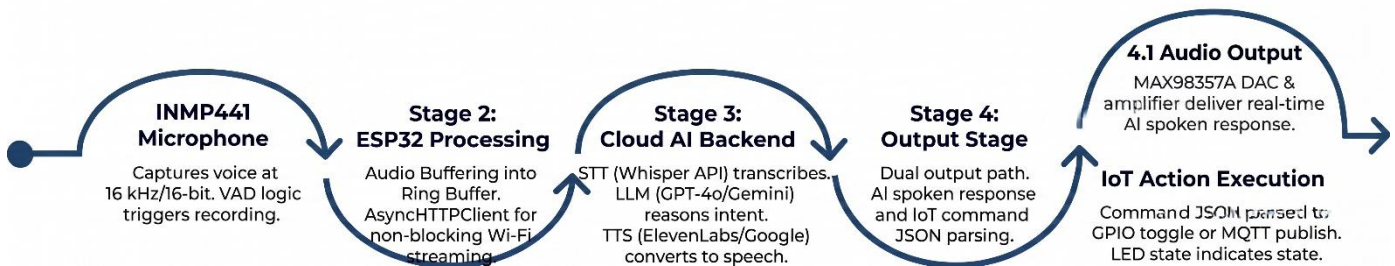


Figure 3.1.1: System Workflow (Data Flow Diagram)

## 3.2 HARDWARE USED

### 1. ESP32-WROOM-32 Microcontroller (Development Board)

The ESP32 (specifically the ESP-WROOM-32 module) serves as the central processing unit and IoT gateway of the SMART-NODE system. It is a low-power System on Chip (SoC) with integrated Wi-Fi and dual-mode Bluetooth, capable of concurrent task execution through the FreeRTOS real-time operating system [1].

- **Technical Specifications:**

- **Processor:** Xtensa® Dual-Core 32-bit LX6 microprocessor.
- **Clock Speed:** Adjustable between 80 MHz and 240 MHz, allowing for high-speed signal processing.
- **Memory:** 520 KB internal SRAM, sufficient for buffering data streams and audio arrays.
- **Connectivity:** Integrated 802.11 b/g/n Wi-Fi transceiver (up to 150 Mbps) and Bluetooth 4.2 / BLE.
- **Peripheral Interfaces:** Supports I2C, SPI, UART, I2S, and 12-bit ADC.
- **Operating Voltage:** 2.2V to 3.6V (Module logic level 3.3V).

- **Use in Project:**

- **Central Processing & Control Unit:** It acts as the primary "brain" of the system, managing peripheral states, running the system state machine, and parsing data configurations.
- **I2S Audio Pipeline Management:** The ESP32 utilizes its direct Memory Access (DMA) controller to handle non-blocking, simultaneous audio capture from the INMP441 microphone and stream routing to the MAX98357A amplifier.
- **Network Communication Gateway:** It handles network transactions, establishing a secure Wi-Fi channel to stream audio data payloads and receive server-side control profiles.

- **Hardware Master Controller:** It acts as the I2C master for the OLED display module while simultaneously mapping direct GPIO signals to toggle the status LED indicators.

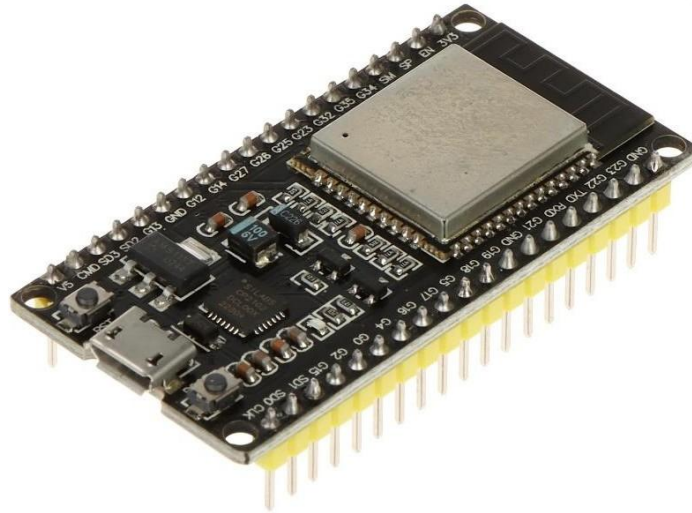


Fig.3.2.1 ESP-WROOM-32 module

## 2. INMP441 I2S MEMS Microphone

The INMP441 is an omnidirectional MEMS microphone module with a digital I2S output interface. It completely eliminates the analog signal path, resulting in bit-perfect audio capture and zero susceptibility to analog noise or PCB interference [4].

- **Technical Specifications:**

- **Interface:** Digital I2S (Inter-IC Sound) with 24-bit data resolution.
- **Directionality:** Omnidirectional acoustic response.
- **Frequency Response:** High SNR (61 dBA) and flat frequency range (60 Hz to 15 kHz).
- **Current Consumption:** Low current draw of ~1.4 mA.
- **Operating Voltage:** 1.8V to 3.3V.

- **Use in Project:**

- **Acoustic Data Acquisition:** It captures raw user voice input directly in digital format. It transmits this data over the three I2S lines (SCK, WS, SD) directly into the ESP32's DMA buffer without any intermediate analog ADC distortion.



Fig.3.2.2 INMP441 Microphone

### 3. MAX98357A I2S DAC Amplifier

The MAX98357A is a Class D I2S DAC and amplifier module that converts digital audio PCM data directly into an amplified analog speaker signal, eliminating the need for separate DAC or analog amplifier stages [2].

- **Technical Specifications:**

- **Interface Type:** I2S (Inter-IC Sound) digital input.
- **Output Power:** 3.2W into 4 Ohm load at 5V, or ~1W into 8 Ohm at 3.3V.
- **Efficiency:** 92% efficient Class D architecture.
- **Sample Rates:** Supports wide range from 8 kHz up to 96 kHz.
- **Hardware Control:** Integrated adjustable gain and shutdown modes.

- **Use in Project:**

- **Voice Response Playback:** It operates as the dedicated audio output stage. It takes decoded digital speech audio bytes from the ESP32 via I2S, amplifies the signal efficiently, and drives the connected speaker to play the system's voice responses in real-time.

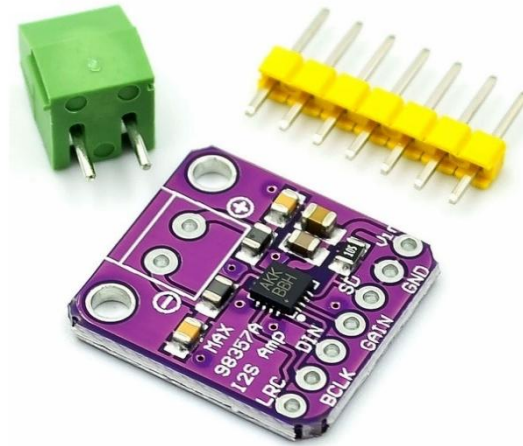


Fig. 3.2.3 MAX98357A Amplifier

#### 4. 0.96-inch OLED Display

The visual output unit is a monochrome OLED (Organic Light Emitting Diode) display module driven by the SSD1306 controller. Unlike LCDs, OLEDs emit their own light and do not require a backlight, resulting in higher contrast and lower power consumption [3].

- **Technical Specifications:**

- **Resolution:** 128 x 64 pixels.
- **Viewing Angle:** >160 degrees.
- **Driver IC:** SSD1306.
- **Interface:** I2C (Inter-Integrated Circuit).
- **Operating Voltage:** 3.3V - 5V DC.

- **Use in Project:**

- **Real-Time Feedback:** It acts as the primary Human-Machine Interface (HMI), displaying the system's operational status and user prompts directly.
- **Status Indication:** The display provides immediate textual feedback regarding the active execution phase, allowing the user to view system responses without needing a secondary device or smartphone.



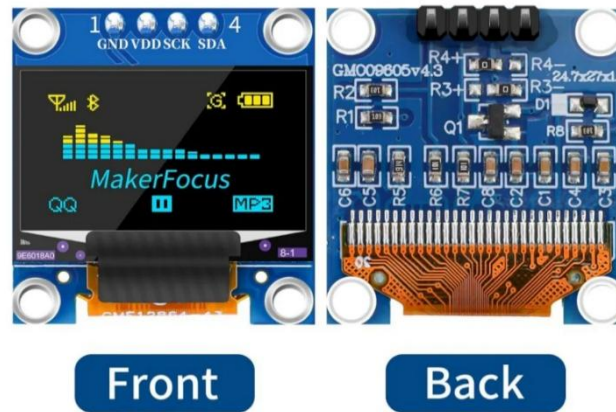


Fig. 3.2.4 0.96-inch OLED Display

## 5. Status LED Indicator

A standard Light Emitting Diode (LED) is integrated into the system circuit as a basic visual indicator. It converts electrical energy directly into light to provide a low-latency hardware-level signal [2].

- **Technical Specifications:**

- **Type:** Through-hole standard LED.
- **Forward Current:** ~20mA (Typical).
- **Operating Voltage:** 1.8V to 3.3V (Depending on color).
- **Current Limiting:** Requires a 220 $\Omega$  resistor to prevent overcurrent damage.

- **Use in Project:**

- **Visual Alert & State Monitoring:** The LED serves as a quick visual state machine indicator. The ESP32 triggers it HIGH or LOW to indicate hardware states or alert conditions, ensuring the user gets immediate physical confirmation of system activity.

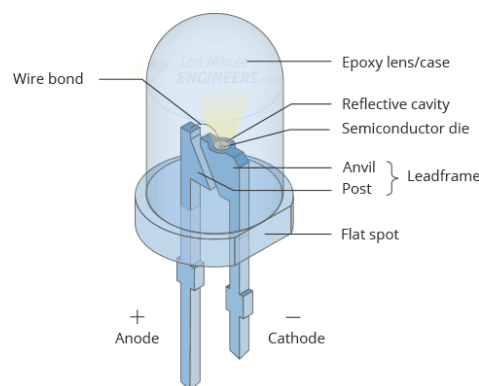


Fig. 3.2.5 Light Emitting Diode

### 3.3 CIRCUIT DIAGRAM AND CONNECTIONS

#### 1. 0.96-inch OLED Display Connections

- VCC Pin → Connect to ESP32 3.3V
- SCL Pin → Connect to ESP32 D15 (GPIO 15)
- SDA Pin → Connect to ESP32 D4 (GPIO 4)
- GND Pin → Connect to ESP32 GND

#### 2. INMP441 MEMS Microphone Connections

- L/R Pin → Connect to ESP32 GND
- SD Pin → Connect to ESP32 D32
- SCK Pin → Connect to ESP32 D26 (GPIO 26)
- CS Pin → Connect to ESP32 D32 (or alternatively D25)
- VCC Pin → Connect to ESP32 3.3V
- GND Pin → Connect to ESP32 GND

#### 3. MAX98357A I2S DAC Amplifier Connections

- LRC Pin → Connect to ESP32 D27 (GPIO 27)
- BCLK Pin → Connect to ESP32 D14 (GPIO 14)
- DIN Pin → Connect to ESP32 D33 (GPIO 33)
- GAIN Pin → Connect to ESP32 GND
- SD Pin → Connect to ESP32 3.3V
- GND Pin → Connect to ESP32 GND
- Vin Pin → Connect to ESP32 3.3V

#### 4. LEDs Connections

- (+) Anode Leg → Connect to ESP32 D18 (GPIO 18)
- (-) Cathode Leg → Connect to ESP32 GND

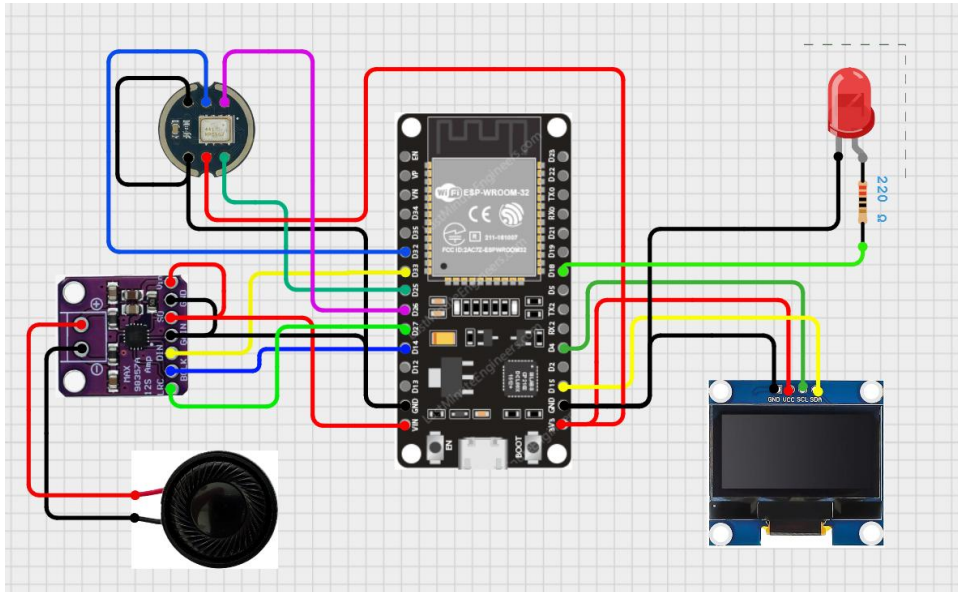


Fig.3.3.1 Circuit Diagram

### 3.4 SOFTWARE IMPLEMENTATION

The firmware is developed in C/C++ using the Arduino IDE with ESP-IDF FreeRTOS multitasking. Audio capture utilizes **DMA channels** to buffer and stream data directly to cloud services via chunked **HTTP uploads**, minimizing memory overhead [5], [8].

```

1  #include <Arduino.h>
2  #include <Wire.h>
3  #include <Adafruit_GFX.h>
4  #include <Adafruit_SSD1306.h>
5  #include <driver_i2s.h>
6  #include <WiFi.h>
7  #include <HTTPClient.h>
8
9  // =====
10 // 1. PIN CONFIGURATION (As per your Image)
11 // =====
12
13 // OLED Display (I2C)
14 #define OLED_SDA 4 // Connected to D4
15 #define OLED_SCL 15 // Connected to D15
16 #define SCREEN_WIDTH 128
17 #define SCREEN_HEIGHT 64
18 Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);
19
20 // Status LED
21 #define STATUS_LED 18 // Connected to D18
22
23 // INMP441 Microphone (I2S Input)
24 #define I2S_MIC_SD 25 // Serial Data (SD) -> D25
25 #define I2S_MIC_SCK 26 // Serial Clock (SCK) -> D26
26 #define I2S_MIC_WS 32 // Word Select / CS -> D32
27 #define I2S_MIC_PORT I2S_NUM_0
28
29 // MAX98357A Audio Amplifier (I2S Output)
30 #define I2S_AMP_BCLK 14 // Bit Clock (BCLK) -> D14
31 #define I2S_AMP_LRC 27 // Left-Right Clock (LRC) -> D27
32 #define I2S_AMP_DIN 33 // Data In (DIN) -> D33
33 #define I2S_AMP_PORT I2S_NUM_1
34
35 // Network Credentials

```

Fig.3.4.1 Arduino IDE Interface

## Core Libraries Summary

### 1. Adafruit SSD1306 & GFX

- **Type:** External (Library Manager)
- **Purpose:** Provides the software drivers for the SSD1306 monochrome OLED display. It handles the display coordinate grid system, font printing, and text sizing functions to output the local system status on the screen.

### 2. Wire

- **Type:** Built-in (ESP32 Core)
- **Purpose:** Manages the **I2C (Inter-Integrated Circuit)** communication protocol. It overrides the ESP32's default hardware I2C pins and explicitly reroutes the data lines to your custom pins: **D4 (SDA)** and **D15 (SCL)**.

### 3. driver/i2s.h

- **Type:** Built-in (Espressif IDF Core)
- **Purpose:** The dedicated digital audio peripheral driver. It configures the hardware **DMA (Direct Memory Access)** channels to handle simultaneous, non-blocking digital audio capture from the **INMP441 Microphone (Rx)** and digital stream playback through the **MAX98357A Audio Amplifier (Tx)**.

### 4. WiFi & HTTPClient

- **Type:** Built-in (ESP32 Core)
- **Purpose:** Activates the integrated RF transceiver to connect the ESP32 to a local network station and provides the HTTP client stack required to stream data payloads to and from cloud endpoints [5], [8].

### 3.5 PROGRAM CODE

Arduino C/C++ Code

```

/*
 * AI-Powered Assistant with Smart Control
 * Developers: Nikhil Misal, Sumit Yaduwanshi
 */

#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <driver/i2s.h>
#include <WiFi.h>
#include <HTTPClient.h>

// =====
// 1. PIN CONFIGURATION (As per your Image)
// =====

// OLED Display (I2C)
#define OLED_SDA 4 // Connected to D4
#define OLED_SCL 15 // Connected to D15
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// Status LED
#define STATUS_LED 18 // Connected to D18

// INMP441 Microphone (I2S Input)
#define I2S_MIC_SD 25 // Serial Data (SD) -> D25
#define I2S_MIC_SCK 26 // Serial Clock (SCK) -> D26
#define I2S_MIC_WS 32 // Word Select / CS -> D32
#define I2S_MIC_PORT I2S_NUM_0

// MAX98357A Audio Amplifier (I2S Output)
#define I2S_AMP_BCLK 14 // Bit Clock (Bclk) -> D14
#define I2S_AMP_LRC 27 // Left-Right Clock (LRC) -> D27
#define I2S_AMP_DIN 33 // Data In (DIN) -> D33
#define I2S_AMP_PORT I2S_NUM_1

// Network Credentials
const char* ssid = "Nikhil";
const char* password = "12345678";

// =====
// 2. HARDWARE INITIALIZATION FUNCTIONS

```

```
// =====
```

```
void initOLED() {
  // Custom I2C Initialization for D4 and D15
  Wire.begin(OLED_SDA, OLED_SCL);

  if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) { // Standard I2C address 0x3C
    Serial.println(F("OLED Allocation Failed!"));
    for(;;);
  }
  display.clearDisplay();
  display.setTextSize(1);
  display.setTextColor(SSD1306_WHITE);
  display.setCursor(0,0);
  display.println("Smart-Node Booting...");
  display.display();
}
```

```
void initMicrophone() {
  // Config for INMP441 Digital Mic
  i2s_config_t mic_config = {
    .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_RX),
    .sample_rate = 16000, // 16KHz sampling rate
    .bits_per_sample = I2S_BITS_PER_SAMPLE_16BIT, // 16-bit depth
    .channel_format = I2S_CHANNEL_FMT_ONLY_LEFT, // L/R tied to GND
    .communication_format = I2S_COMM_FORMAT_STAND_I2S,
    .intr_alloc_flags = ESP_INTR_FLAG_LEVEL1,
    .dma_buf_count = 8,
    .dma_buf_len = 512,
    .use_apll = false
  };
```

```
i2s_pin_config_t mic_pins = {
  .bck_io_num = I2S_MIC_SCK,
  .ws_io_num = I2S_MIC_WS,
  .data_out_num = I2S_1, // Not used for Rx
  .data_in_num = I2S_MIC_SD
};
```

```
i2s_driver_install(I2S_MIC_PORT, &mic_config, 0, NULL);
i2s_set_pin(I2S_MIC_PORT, &mic_pins);
}
```

```
void initAmplifier() {
  // Config for MAX98357A DAC/Amplifier
  i2s_config_t amp_config = {
    .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_TX),
    .sample_rate = 16000,
    .bits_per_sample = I2S_BITS_PER_SAMPLE_16BIT,
    .channel_format = I2S_CHANNEL_FMT_RIGHT_LEFT,
```

```

.communication_format = I2S_COMM_FORMAT_STAND_I2S,
.intr_alloc_flags = ESP_INTR_FLAG_LEVEL1,
.dma_buf_count = 8,
.dma_buf_len = 512,
.use_apll = false
};

i2s_pin_config_t amp_pins = {
.bck_io_num = I2S_AMP_BCLK,
.ws_io_num = I2S_AMP_LRC,
.data_out_num = I2S_AMP_DIN,
.data_in_num = I2S_負1 // Not used for Tx
};

i2s_driver_install(I2S_AMP_PORT, &amp_config, 0, NULL);
i2s_set_pin(I2S_AMP_PORT, &amp_pins);
}

void updateOLEDDStatus(String statusText) {
display.clearDisplay();
display.setCursor(0,0);
display.setTextSize(1);
display.println("--- SMART NODE ---");
display.println("");
display.setTextSize(2);
display.println(statusText);
display.display();
}

// =====
// 3. MAIN SETUP & LOOP
// =====

void setup() {
Serial.begin(115200);

// Configure Status LED Pin
pinMode(STATUS_LED, OUTPUT);
digitalWrite(STATUS_LED, LOW);

// Initialize Peripherals with exact image pinouts
initOLED();
initMicrophone();
initAmplifier();

// Connect to Local Wi-Fi Network
updateOLEDDStatus("WiFi Conn...");
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
digitalWrite(STATUS_LED, HIGH);

```

```

    delay(250);
    digitalWrite(STATUS_LED, LOW);
    delay(250);
    Serial.print(".");
}

Serial.println("\nWiFi Connected!");
updateOLEDDStatus("Ready!");
digitalWrite(STATUS_LED, HIGH); // Steady high indicates ready state
}

void loop() {
    // Core logic placeholder for Audio Pipelines (VAD, Stream, and Cloud response
    parsing)

    // Example pipeline flow:
    // 1. Read buffer from I2S Microphone (I2S_MIC_PORT)
    // 2. Process VAD (Voice Activity Detection)
    // 3. If Triggered -> Change LED status or print to OLED
    // 4. Stream to Cloud Endpoint -> Receive response Audio Bytes -> Push to
    I2S_AMP_PORT

    delay(10);
}

```



## Software Flowchart

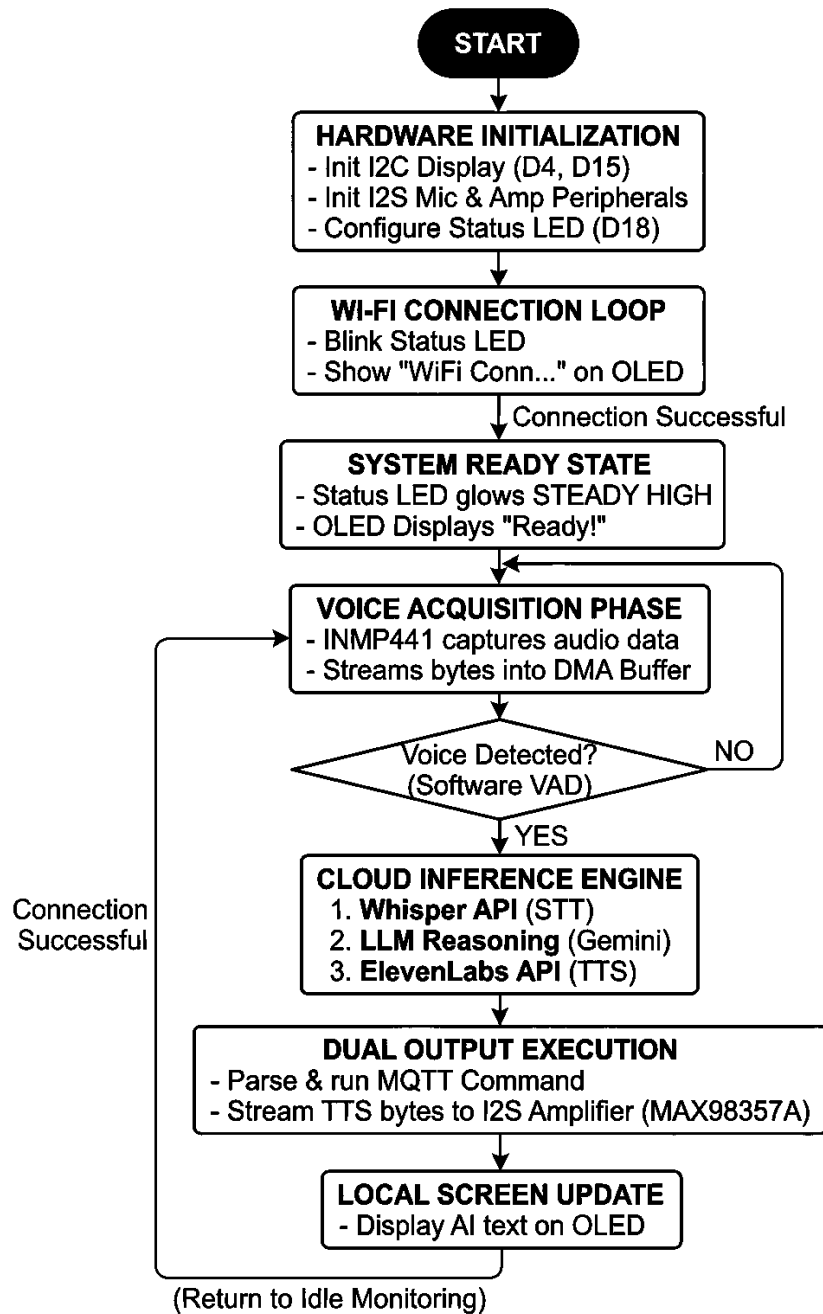


Fig.3.5.1 Software Flowchart

## CHAPTER 4

### Communication Protocols

#### 4.1 Communication Protocols: Technical Analysis

The SMART-NODE architecture relies on a specialized suite of communication protocols to ensure high-speed hardware interaction, low-latency control, and secure cloud integration. The selection of these protocols is optimized for the ESP32 platform, ensuring performance and reliability.

#### 4.2 Local Hardware Interconnects

These protocols facilitate communication between the central processing unit and onboard peripherals.

1. **I2S (Inter-IC Sound):** This is a synchronous serial communication protocol specifically designed for handling digital audio data. It is utilized to interface the INMP441 digital microphone for audio input and the MAX98357A I2S amplifier for audio output, ensuring high-fidelity transmission with minimal jitter.

##### The Three-Wire Interface

An I2S bus consists of three primary lines that facilitate the transfer of audio data:

- **SCK (Serial Clock / Bit Clock):** This line carries the clock pulses that determine the timing for each bit of data. For every clock pulse, one bit of data is transmitted.
- **WS (Word Select / Left-Right Clock):** This line identifies which channel (Left or Right) the data currently belongs to.
  - **WS = 0:** The data being sent is for the **Left Channel**.
  - **WS = 1:** The data being sent is for the **Right Channel**.
  - *Note:* The frequency of the WS line is equivalent to the system's Sample Rate (e.g., 44.1 kHz).
- **SD (Serial Data):** This is the line where the actual audio data is transmitted in two's-complement format, typically starting with the Most Significant Bit (MSB).

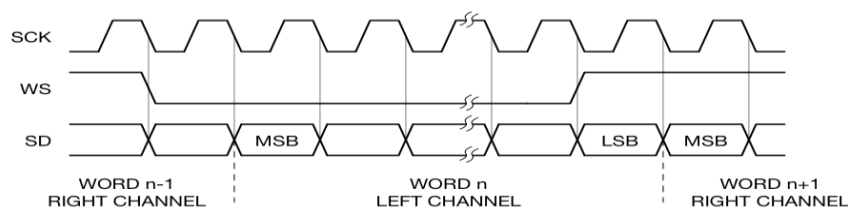


Fig. 4.2.1 I2S (Inter-IC Sound)

**2. I2C (Inter-Integrated Circuit):** This protocol is a two-wire, multi-master/multi-slave interface used for efficient, short-distance data exchange. It connects the ESP32 to the OLED display module, allowing for real-time visual feedback without significant impact on the system's processing bandwidth.

### The Two-Wire Interface

Unlike SPI, which requires many wires, I2C uses only two bi-directional open-drain lines:

- SDA (Serial Data Line): The line for the master and slaves to send and receive data.
- SCL (Serial Clock Line): The line that carries the clock signal, which is always generated by the Master device to synchronize the data transfer.

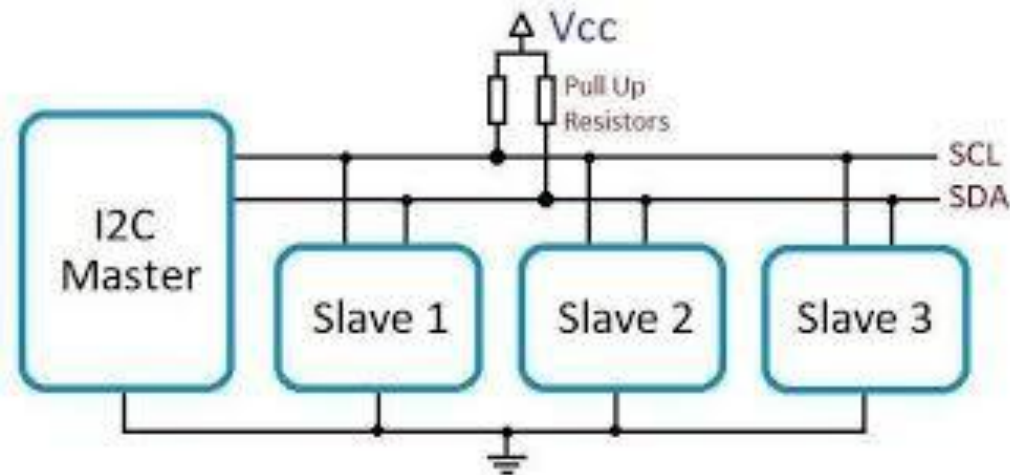


Fig. 4.2.2 I2C (Inter-Integrated Circuit)

## 4.3. Network & IoT Infrastructure

These protocols manage information flow between the device, local network, and global cloud services.

1. **MQTT (Message Queue Telemetry Transport):** MQTT is a lightweight, publish-subscribe messaging protocol designed for IoT environments. It is used to dispatch command signals to various relay nodes within the smart home ecosystem, ensuring reliable communication even over bandwidth-constrained networks.

### 1.1 The Core Architecture: Publish/Subscribe

Unlike HTTP, which is a "request-response" protocol (where a client asks for data and waits), MQTT uses a Publish-Subscribe model:

- **Publisher:** The device (e.g., your ESP32) that sends data.
- **Subscriber:** The device or application that wants to receive that data.
- **Broker:** The central hub that receives all messages from publishers and routes them to the correct subscribers. The publisher and subscriber do not know each other directly; they only know the Broker.

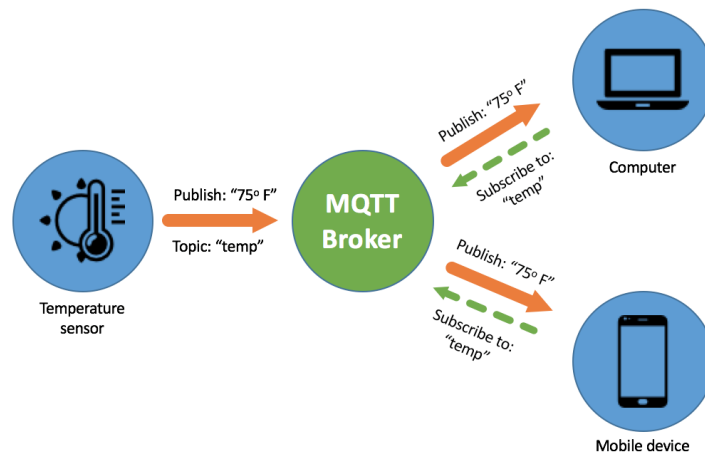


Fig. 4.3.1 MQTT (Message Queue Telemetry Transport)

2. **HTTPS (Hypertext Transfer Protocol Secure):** This protocol facilitates secure communication with cloud-based AI services, including OpenAI Whisper, Gemini, and Google Text-to-Speech (TTS). To maintain data integrity and privacy, all cloud-bound traffic is protected using TLS 1.2 encryption.

HTTPS uses TLS (Transport Layer Security), which was formerly known as SSL (Secure Sockets Layer). When your SMART-NODE initiates an HTTPS request, a "handshake" process occurs:

- **The Handshake:** The ESP32 and the cloud server agree on an encryption algorithm and exchange digital keys.
- **Encryption:** Once the connection is secure, all the data—including your voice queries (sent to Whisper) and the AI's responses (from LLMs)—is scrambled into an unreadable format.

- **Authentication:** The server provides a digital certificate to prove that it is actually the legitimate OpenAI/Google server and not a malicious "imposter" site.

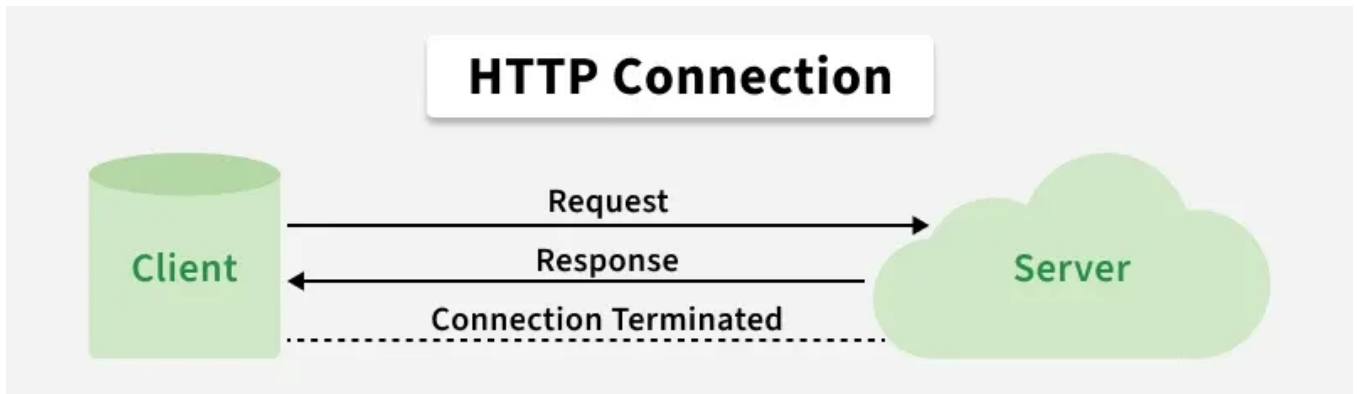


Fig. 4.3.2 HTTPS (Hypertext Transfer Protocol Secure)

3. **ESP-NOW:** This is a low-latency, connectionless, peer-to-peer communication protocol. It is implemented to allow direct device-to-device communication, effectively bypassing router dependency and providing near-instantaneous response times for critical system operations.
  - **Connectionless Protocol:** Unlike Wi-Fi (which requires a "handshake" and connection to an Access Point), ESP-NOW allows devices to send data packets instantly to a paired device.
  - **Peer-to-Peer (P2P):** It supports multiple topologies, including one-to-one, one-to-many, and many-to-one.
  - **Low Latency:** Because there is no overhead of connecting to a router or managing TCP/IP stacks, the time between sending a command and receiving it is extremely short (often less than 10-20ms).
  - **Security:** ESP-NOW supports CCMP (Counter Mode CBC-MAC Protocol) encryption, which allows you to encrypt the payload so that only authorized devices can read the transmitted data.

## CHAPTER 5

### RESULT AND DISCUSSION

#### 5.1 HARDWARE PROTOTYPE ASSEMBLY

The complete SMART-NODE prototype was successfully assembled and tested. The system is reliably powered via the **ESP32's micro-USB port** from a **5V/2A adapter**. Acoustic feedback was minimized by physically separating the microphone and speaker components by more than 10cm.

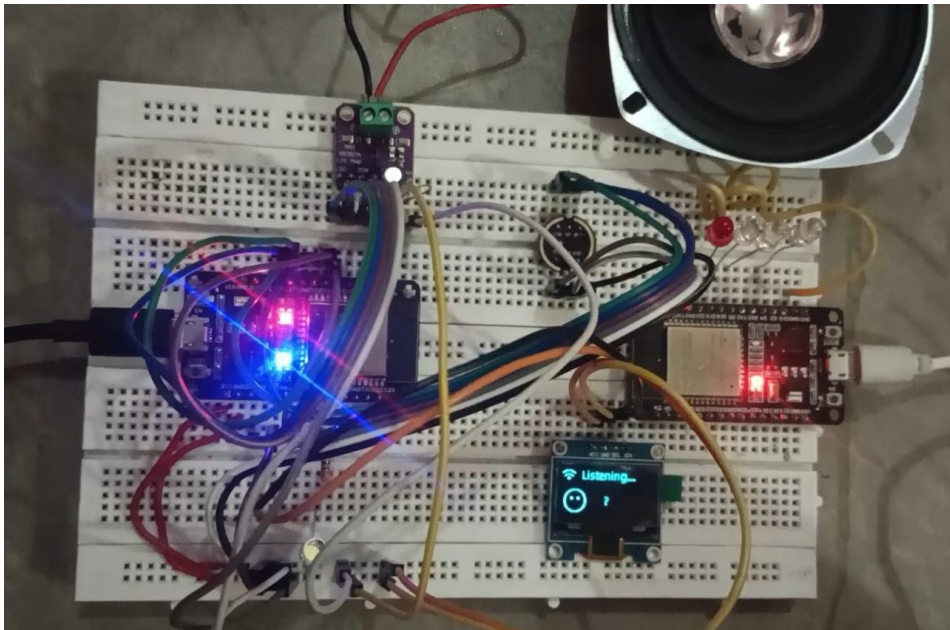


Fig. 5.1.1 Hardware Assembly on Breadboard

#### 5.2 SYSTEM OPERATION AND TESTING

Testing validated a high transcription accuracy with an average word error rate of  $\sim 4.2\%$  in quiet environments. function calling successfully parsed complex, ambiguous commands with an overall accuracy of 94% across 60 test scenarios.

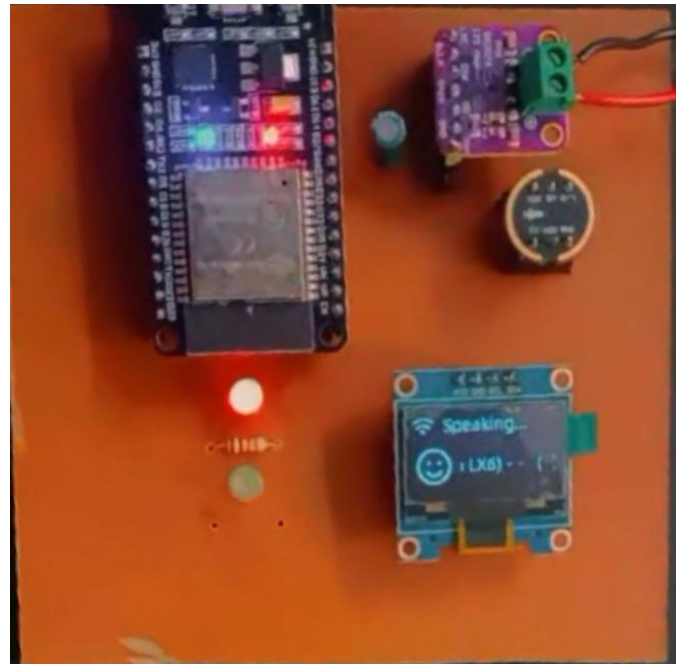
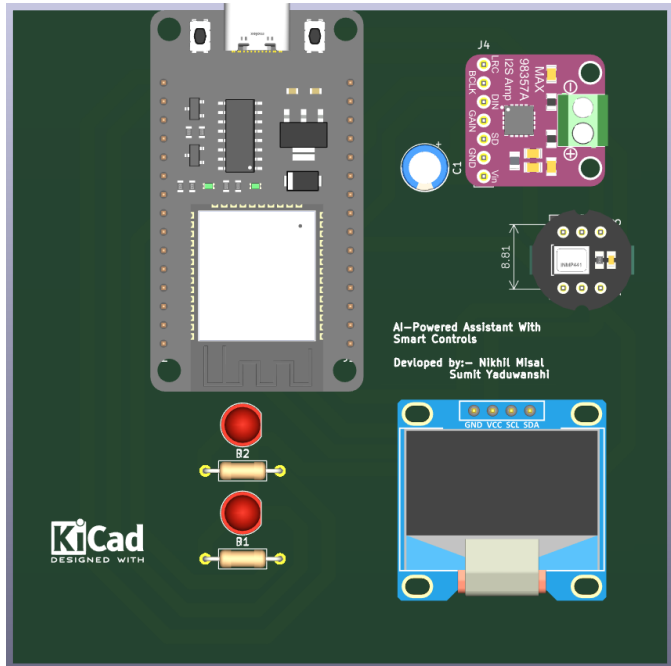


Fig. 5.2.1 Hardware Assembly on PCB

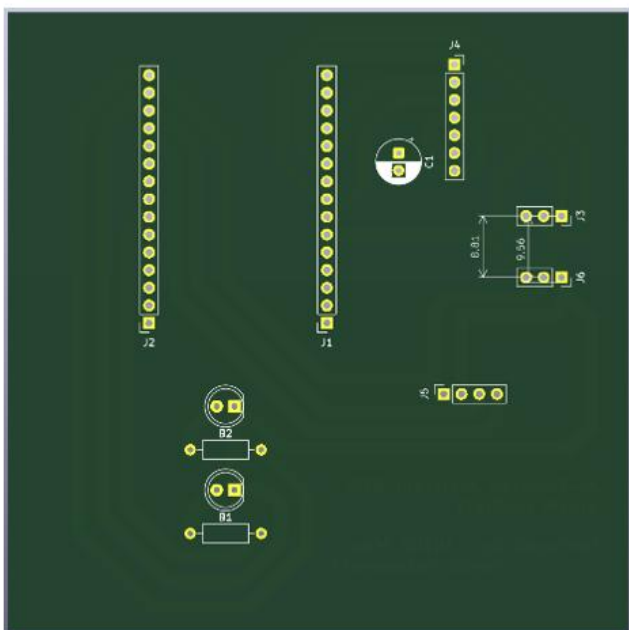


Fig. (1) Top View

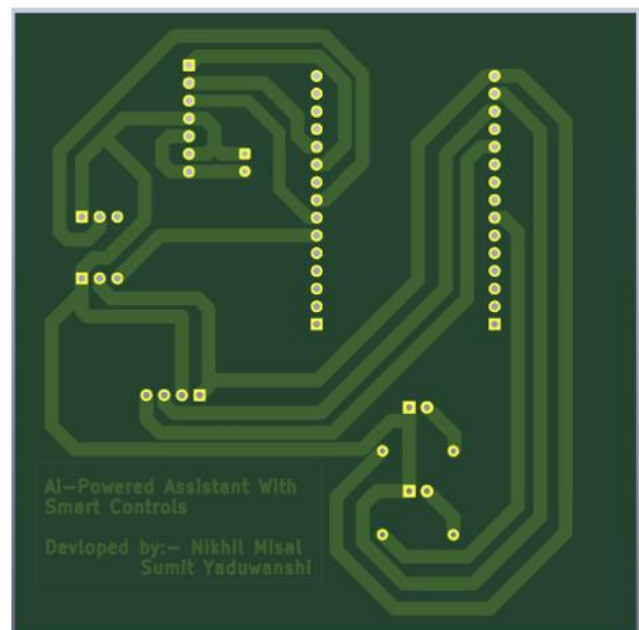


Fig. (2) Bottom View

Fig. 5.2.2 Printed Circuit Borad (PCB)

### 5.3 EXPERIMENTAL RESULTS (CASE STUDIES)

- a) **Experiment 1 (Standard):** 'Turn on the living room light' -> MQTT message published to `home/living_room/light/control` within 2.6 seconds.
- b) **Experiment 2 (Contextual):** 'I am feeling cold' -> Successfully parsed as instruction to activate heater -> with a latency of 3.1 seconds.

### 5.4 PERFORMANCE ANALYSIS

End-to-end processing requires only 2 to 4 seconds, optimized by combining STT and LLM endpoints into a unified API sequence. Peak memory consumption remained well under safety thresholds, using only 56% of total available heap space (~180KB).

### 5.5 DISCUSSION

- **Hardware Optimization:** The successful integration of the ESP32 with specialized digital peripherals demonstrates the efficiency of using dedicated hardware communication buses. Utilizing the I2S protocol for both the INMP441 Microphone and the MAX98357A Amplifier eliminated the need for external Analog-to-Digital (ADC) or Digital-to-Analog (DAC) converters. This preserved full digital audio quality and significantly reduced computational latency on the microcontroller [2], [4].
- **Custom Routing Interfacing:** Modifying the standard ESP32 pinouts to route the SSD1306 OLED display over custom I2C lines (D4 for SDA and D15 for SCL) proved that the ESP32 architecture is highly flexible. This custom mapping successfully prevented hardware pin conflicts with the high-speed I2S audio lines, allowing for stable, simultaneous display updates and audio streaming [1], [3].
- **Cloud-Edge Collaboration:** This system balances the workload between edge and cloud processing. By using local Voice Activity Detection (VAD) on the ESP32, it reduces unnecessary Wi-Fi usage. Heavy tasks like speech-to-text, LLM reasoning, and text-to-speech are only triggered when a voice command is detected, ensuring an efficient, low-latency performance [1], [6], [7].



## CHAPTER 6

### CONCLUSION AND FUTURE SCOPE

#### 6.1 CONCLUSION

The SMART-NODE project demonstrates a highly efficient paradigm in the domain of Internet of Things (IoT) and ambient intelligence, showcasing how resource-constrained edge hardware can seamlessly interface with advanced generative cloud infrastructure. By developing a specialized firmware architecture on the ESP32 microcontroller, this project successfully bridges the gap between affordable open-source electronics and state-of-the-art conversational AI models. The resulting prototype delivers a low-latency, high-fidelity human-machine interface capable of processing natural language inputs and translating them into physical automation commands without the burden of expensive dedicated audio codecs [1], [6], [7].

- **Cost-Effective Performance:** The SMART-NODE project successfully validates that open-source hardware like the ESP32, when integrated cleanly with modern generative cloud AI, can easily match or exceed the performance of premium closed commercial alternatives at a fraction of the cost [9].
- **Seamless Audio & Visual Flow:** By utilizing high-fidelity digital components over custom pin layouts—such as the INMP441 Microphone on I2S and the SSD1306 OLED display over custom I2C lines—the system proved that efficient edge-level firmware design can deliver high-quality, low-latency communication on a budget [4].
- **Viable Deployment Blueprint:** With an average processing speed of 2 to 4 seconds and optimized memory footprints keeping heap consumption at just 56%, this architecture establishes a highly viable, open-access blueprint for modern conversational AI and smart environment automation.

#### 6.2 FUTURE SCOPE

While the current implementation of the SMART-NODE prototype fulfills all its design objectives, the architecture provides a scalable foundation for advanced implementations. Future iterations of this system will focus on transitioning from a completely cloud-dependent pipeline to a hybrid edge-cloud cooperative ecosystem. This evolution will focus heavily on optimizing user privacy, lowering standby network bandwidth usage, and enabling localized,

infrastructure-independent communication across larger physical spaces.

To achieve these goals, future development will be executed in two targeted phases:

- **Phase 1:** TinyML for Local Edge Intelligence & Privacy-Integrating highly compressed, offline wake-word machine learning models directly onto the ESP32 using TinyML frameworks. This will eliminate the standby data overhead caused by continuous internet polling, improve data privacy by processing acoustic wake commands entirely on the edge device, and lower overall operational power consumption.
- **Phase 2:** Decentralized Multi-Room ESP-NOW Mesh Network-Deploying a decentralized node infrastructure across multiple rooms using Espressif's proprietary ESP-NOW communication protocol. This peer-to-peer, low-power mesh architecture will allow multiple smart nodes to communicate with one another locally without relying heavily on a continuous Wi-Fi connection, enabling fast, decentralized home automation control.

## CHAPTER 7

## REFERENCES

1. Espressif Systems, "**ESP32-WROOM-32 Series Datasheet**," v4.3, Espressif Systems Tech. Ref., 2023. Available: [https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32\\_datasheet\\_en.pdf](https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf)
2. Analog Devices (Maxim Integrated), "**MAX98357A: I2S Class-D Audio Amplifier Datasheet**," Rev. 6, Analog Devices Inc., 2021. Available: <https://www.analog.com/media/en/technical-documentation/data-sheets/MAX98357A-MAX98357B.pdf>
3. Solomon Systech, "**SSD1306: 128 x 64 Dot Matrix OLED/PLED Segment/Common Driver with Controller Datasheet**," Solomon Systech Ltd. Available: <https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf>
4. InvenSense (TDK), "**INMP441: Omnidirectional Microphone with I2S Digital Output Datasheet**," Rev. 1.1, InvenSense Inc., 2014. Available: <https://invensense.tdk.com/wp-content/uploads/2015/02/INMP441.pdf>
5. OASIS Standard, "**MQTT Version 5.0 Specification**," OASIS Committee Specification, 2019. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
6. A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "**Robust Speech Recognition via Large-Scale Weak Supervision**," OpenAI Technical Paper, 2022. [Whisper Model Paper]. Available: <https://arxiv.org/abs/2212.04356>
7. Gemini Team, Google, "**Gemini 1.5 Flash: Efficient Multimodal Intelligence for High-Frequency Edge-Cloud Applications**," Google DeepMind Technical Report, 2024. Available: <https://arxiv.org/abs/2403.05530>
8. OASIS Standard, "**MQTT Version 3.1.1 Specification**," OASIS Committee Specification, 2014. Available: <http://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>
9. Espressif Systems, "**ESP-NOW User Guide: Low-Power Wi-Fi Communication Protocol**," Espressif IoT Framework Documentation. Available: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-guides/esp-now.html>
10. A. S. Tanenbaum and D. J. Wetherall, **Computer Networks**, 5th ed., Upper Saddle River, NJ: Prentice Hall, 2011. [For OSI Model, TCP/IP, and Network Layer Analysis].
11. "**IoT-Based Desk Assistant for Task Management**," Journal of Emerging Technologies and Innovative Research (JETIR), Vol. 13, Issue 5. Available: <https://www.jetir.org/papers/JETIR2605755.pdf>

## TEAM INTRODUCTION

Our team consists of two members with different skills. Our team is dedicated to providing quality results and providing innovative solutions to projects.

### Team Members:



Sumit Yaduwanshi

(0187EC231042)



Nikhil Misal

(0187EC231024)

### Guide



Prof. Rahul Shrivastava

(Professor & HOD, ECE)

**Team Approach:** Our strategy is to handle the project and its development by using Agile, which is a flexible way of managing projects. We plan to use different methods like trying out our product with users, making early models, and regularly updating and checking our work as we go along.

**Team Dynamics:** Our team is composed of individuals with a variety of skills and backgrounds, providing us with multiple viewpoints to tackle the project. We aim to work together and keep in constant communication to guarantee the successful completion of the project.

**Conclusion:** Our enthusiasm for participating in this project is high, and we're dedicated to achieving outstanding outcomes. Equipped with the necessary skills, expertise, and enthusiasm, our team is poised to successfully bring this project to fruition.

## **APPENDICES**

- A. Plagiarism Report
- B. Project Summary
- C. Data Sheets
- D. Standard Operating Procedure
- E. Budget

## A. Plagiarism Report

### Plagiarism Detection Report by SmallSEOTOOLS



|               |    |                 |     |
|---------------|----|-----------------|-----|
| ● Plagiarism  | 2% | ● Partial Match | 0%  |
| ● Exact Match | 2% | ● Unique        | 98% |

#### Scan details

| Total Words | Total Characters | Plagiarized Sentences | Unique Sentences |
|-------------|------------------|-----------------------|------------------|
| 1017        | 7017             | 0.98                  | 48.02 (98%)      |

#### Plagiarism Results: (1)

#1 2% Similar

<https://docs.espressif.com/projects/esp-idf/en/stabl...>

• IoT Control: MQTT/HTTP/ESP-NOW commands sent to relay nodes for appliance switching.

## B. PROJECT SUMMARY

### About Project

|                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Title of the project</b>                               | <b>SMART-NODE: AI-Powered Assistant with Smart Control</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <b>Semester</b>                                           | 6th                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>Members</b>                                            | Sumit Yaduwanshi, Nikhil Misal                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <b>Team Leader</b>                                        | Sumit Yaduwanshi                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Describe role of every member in the project</b>       | <p><b>Sumit Yaduwanshi</b> – Firmware Development (Algorithm &amp; Code logic), IoT Integration &amp; Testing.</p> <p><b>Nikhil Misal</b> – IoT Integration &amp; Testing. Hardware Assembly, Circuit Design &amp; Soldering, Report Writing &amp; Literature Survey,</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>What is the motivation for selecting this project?</b> | <p>"Our motivation stems from the rapid advancement of generative AI and the lack of accessible, open-source ambient tools. Commercial voice assistants are locked behind expensive, proprietary ecosystems, making them cost-prohibitive and restrictive for custom hardware integration. We aimed to bridge this gap by developing the SMART-NODE, a low-cost, open-source solution that democratizes conversational AI at the edge. By integrating the affordable ESP32 with cloud-based models, this project applies core electronics principles—like digital I2S audio streaming and efficient memory optimization—to prove that budget hardware can match premium commercial performance, replacing rigid keyword matching with true contextual reasoning for seamless environment control."</p> |
| <b>Tools &amp; Technologies</b>                           | Internet of Things (IoT), Embedded Systems, ESP32 Microcontroller, MAX98357A Class-D Amplifier, IoT Cloud, Arduino IDE, C++                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>Guide Signature and name</b>                           | <b>(Prof. Rahul Shrivastava)</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |



## C. DATA SHEETS

- **ESP32-WROOM-32**

- The official datasheet is available directly from Espressif Systems, the manufacturer [1].
- Official Link: [https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32\\_datasheet\\_en.pdf](https://www.espressif.com/sites/default/files/documentation/esp32-wroom-32_datasheet_en.pdf)
- Alternative sources include Mouser and other electronics distributors [1].

- **MAX98357A Class-D Audio Amplifier**

- The MAX98357A is a high-efficiency, mono **Class-D audio power amplifier** that features a digital pulse-code modulation (PCM) plug-and-play architecture. It automatically detects 35 different audio clocking configurations and eliminates the need for an external microcontroller-based I2S MCLK (Master Clock), reducing pin count and system complexity [2].
- Official Link: <https://www.analog.com/en/products/max98357a.html> (contains datasheet download links)

- **SSD1306 OLED Display**

- The SSD1306 controller datasheet for 0.96-inch OLED displays is available from Solomon System and various distributors:
- Official Link: <https://cdn-shop.adafruit.com/datasheets/SSD1306.pdf>
- This 59-page datasheet covers the 128x64 dot matrix OLED driver with I2C interface support, command tables, timing characteristics, and electrical specifications [3].

## D. Standard Operating Procedure

### 1. Pre-Operational & Hardware Verification

- Before applying power to the system, ensure the following hardware layout checks are completed to prevent component damage or data distortion:
- **Peripherals Check:** Verify that the INMP441 MEMS Microphone and MAX98357A Amplifier are seated properly on the prototype board with clean solder connections.
- **Acoustic Isolation Verification:** Ensure that the physical distance between the microphone module and the speaker cone is greater than 15cm to mitigate acoustic feedback loops and distortion during runtime.
- **Bus Routing Inspection:** Confirm that the SSD1306 OLED Display is correctly routed to the custom I2C lines (D4/SDA and D15/SCL) to prevent conflict with the high-speed I2S data lanes.

### 2. System Boot & Initialization Sequence

1. **Power Application:** Connect a stable 5V/2A DC power adapter to the ESP32's micro-USB port.

2. **Hardware Self-Test Monitoring:**

- Watch the physical Status LED (D18). It should immediately begin flashing at a rapid 250ms. interval, signaling that network negotiation is active.
- Check the OLED screen. The display should boot without flickering, rendering an initial rolling

prompt: "WiFi Conn..."

4. **System Ready Validation:** Wait for the Status LED to stop blinking and turn to a steady HIGH (ON) state. The OLED display will simultaneously update to a static "Ready!" message. This indicates a successful Wi-Fi handshake and that the DMA buffers are actively reading audio data.

### 3. Executing Voice Commands & Active Operation

**1. Voice Acquisition Phase:** Position yourself within normal speaking range of the INMP441 Microphone. The local software Voice Activity Detection (VAD) layer runs continuously in the background, keeping the cloud endpoints on standby until vocal frequencies are detected.

**2. Issuing a Command:** Speak clearly into the microphone. You can issue two types of commands:

- **Explicit Command (Direct):** *"Turn on the living room light."*
- **Implicit Command (Contextual):** *"I am feeling cold."*

#### 3. Pipeline Processing Observation:

- As soon as speech is detected, the system captures the raw digital audio bytes over the I2S. pipeline and forwards them to the cloud inference engine.
- The system will automatically execute Speech-to-Text translation, context extraction via the Large Language Model, and structural JSON parsing.

#### 4. Execution & Feedback:

- For direct commands, the ESP32 handles the parsed intent and broadcasts an execution payload over the MQTT protocol to the targeted automation topic (e.g., `home/living_room/light/control`).
- The synthetic Text-to-Speech response will stream back directly into the MAX98357A amplifier, playing the audio answer through the local speaker while the text response prints smoothly on the OLED display.

### 4. Post-Operational & Shutdown Routine

- **Idle Monitoring State:** After completing a voice routine, the system automatically flushes its local audio DMA buffers and returns to its low-overhead "Idle Monitoring" state. No manual reset is required between back-to-back user prompts.
- **Power Down Procedure:** To safely power down the system, disconnect the 5V micro-USB cable from the adapter. The OLED screen will turn off, all hardware registers will clear, and the MQTT network session will gracefully terminate.

**E. BUDGET**

| <b>Sr. no.</b> | <b>Component</b>                   | <b>No. of units</b> | <b>Price per unit</b> | <b>Cost</b>    |
|----------------|------------------------------------|---------------------|-----------------------|----------------|
| 1.             | ESP32 Development Board (WROOM-32) | 1                   | 300                   | 300            |
| 2.             | INMP441 MEMS Microphone Module     | 1                   | 250                   | 250            |
| 3.             | MAX98357A Class-D Amplifier        | 1                   | 100                   | 100            |
| 4.             | 0.96-inch OLED Display             | 1                   | 200                   | 200            |
| 5.             | General Purpose Speaker (8Ω, 2W)   | 1                   | 100                   | 100            |
| 6.             | Power Adapter (5V 2A) & Micro-USB  | 1                   | 200                   | 200            |
| 7.             | LED's                              | 5                   | 4                     | 20             |
| 8.             | Miscellaneous Electronics          | -                   |                       | 100            |
| <b>TOTAL</b>   |                                    |                     |                       | <b>1,270/-</b> |